AI ASSISTED CODING

Assignment-10.3

Name: A. sai shrehan

HTno:2303A52299

Batch:45

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to

calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:

1. Run the code and test it with factorial(5).

2. Use an AI assistant to:

o Identify the logical bug in the code.

o Explain why the bug occurs (e.g., off-by-one error).

o Provide a corrected version.

3. Compare the AI's corrected code with your own manual fix.

4. Write a brief comparison: Did AI miss any edge cases (e.g.,

negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

```python
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result
factorial(5)
```

Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show th

Comment 1 of 1

Code Review

The call to `factorial(5)` does not display or use the result. Consider printing the result or assigning it to a variable for clarity.

No change found to suggest.

Apply Discard

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL COMMENTS ···

Filter (e.g. text, author)

assignment_10_3.py assignment 10.3

Code Review · The call to `factorial(5)` does not display or use the result. Consider printing the result or assigning it ... [Ln 6]

task6.py U | Assignment_3-5.py U | assignment_10-3.py U

assignment_10-3.py > ...

```python
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result
print(factorial(5))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_10-3.py
120
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:\Users\Sushri\Documents\3-2\AI-Assisted_coding\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task 2 — Improving Readability & Documentation

Scenario:The following code works but is poorly written:

.

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
```

Instructions:

5. Use AI to:

o Critique the function’s readability, parameter naming, and lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

```python
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        return a / b
```

Pin selection to current chat prompt (Ctrl+Alt+X) | Don't

Comment 1 of 4

Code Review

Division by zero is not handled in the `div` case. You should check if $b \neq 0$ before performing division to avoid a runtime error.

No change found to suggest.

Keep Undo

```python
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        return a / b
def perform_arithmetic_operation(first_number, second_number, operation):
    """
    Perform a basic arithmetic operation on two numbers.

    This function takes two numeric operands and an operation string,
    then returns the result of the specified arithmetic operation.
    Supports addition, subtraction, multiplication, and division.

    Parameters:
    -----------
    first_number : int or float
        The first operand for the arithmetic operation.
    second_number : int or float
        The second operand for the arithmetic operation.
    operation : str
        The arithmetic operation to perform. Must be one of:
        - "add": Addition (first_number + second_number)
        - "sub": Subtraction (first_number - second_number)
        - "mul": Multiplication (first_number * second_number)
        - "div": Division (first_number / second_number)

    Returns:
    --------
```

Keep Undo 1 of 1

ent 10.3 > assignment_10_3.py > perform_arithmetic_operation

```python
def perform_arithmetic_operation(first_number, second_number, operation):
    Raises:
    -------
    ValueError
        If operation is not one of the valid strings: "add", "sub", "mul", "div".
    ZeroDivisionError
        If operation is "div" and second_number is zero.
    TypeError
        If first_number or second_number are not numeric types.

    Examples:
    ---------
    >>> perform_arithmetic_operation(10, 5, "add")
    15
    >>> perform_arithmetic_operation(10, 5, "sub")
    5
    >>> perform_arithmetic_operation(10, 5, "mul")
    50
    >>> perform_arithmetic_operation(10, 5, "div")
    2.0

    >>> perform_arithmetic_operation(10, 0, "div")
    Traceback (most recent call last):
        ...
    ZeroDivisionError: Cannot divide by zero. Second operand must be non-zero.
    """

    # Input validation: Check types
    if not isinstance(first_number, (int, float)) or not isinstance(second_number,
        raise TypeError(
            f"Operands must be numeric (int or float). "
            f"Got: first_number={type(first_number).__name__}, "
            f"second_number={type(second_number).__name__}"
        )

    # Input validation: Check operation is valid
    valid_operations = {"add", "sub", "mul", "div"}
```

Keep Undo 1 of 1

```python
    # Input validation: Check operation is valid
    valid_operations = {"add", "sub", "mul", "div"}
    if operation not in valid_operations:
        raise ValueError(
            f"Invalid operation '{operation}'. "
            f"Must be one of: {', '.join(sorted(valid_operations))}"
        )

    # Perform the operation
    if operation == "add":
        return first_number + second_number
    elif operation == "sub":
        return first_number - second_number
    elif operation == "mul":
        return first_number * second_number
    elif operation == "div":
        if second_number == 0:
            raise ZeroDivisionError(
                "Cannot divide by zero. Second operand must be non-zero."
            )
        return first_number / second_number
```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer

submits:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
        return True
```

Instructions:

8. Verify the function works correctly for sample inputs.

9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter

with AI explanation) to:

o List all PEP8 violations.

o Refactor the code (function name, spacing, indentation,

naming).

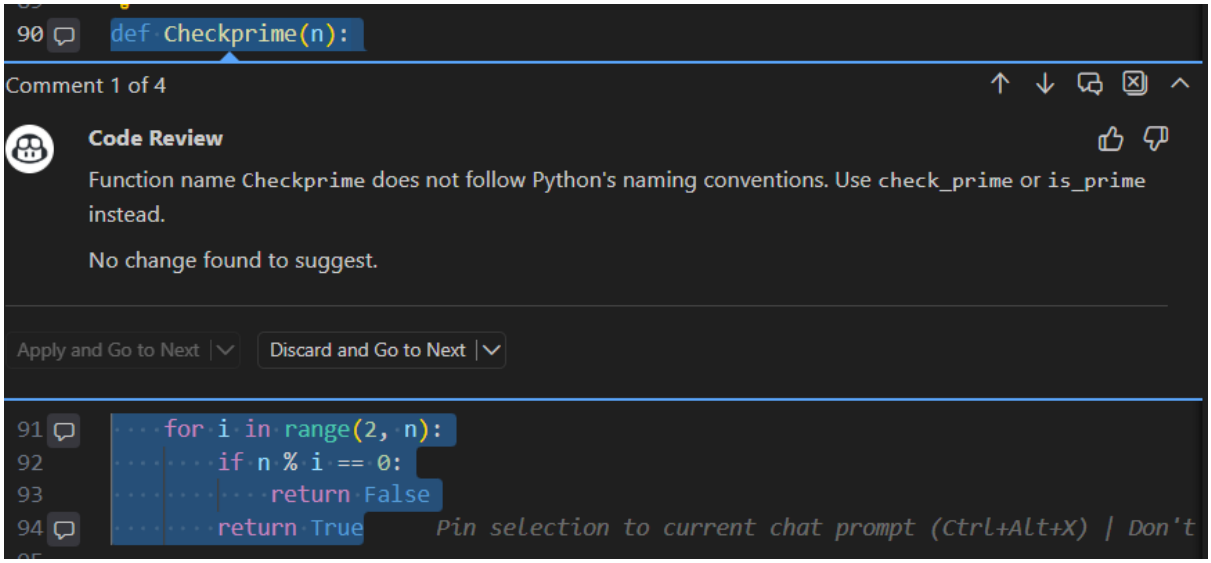
10. Apply the AI-suggested changes and verify functionality is preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
        return True
```



Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

o Readability and naming.

o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

```python
def check_Prime(number: int) -> bool:
    """
    Determine whether an integer is prime.

    Parameters
    ----------
    number : int
        The integer to test for primality.

    Returns
    -------
    bool
        `True` if `number` is prime, otherwise `False`.

    Raises
    ------
    TypeError
        If `number` is not an `int`.

    Examples
    --------
    >>> check_Prime(2)
    True
    >>> check_Prime(15)
    False
    """
    if not isinstance(number, int):
        raise TypeError("Input must be an int")

    if number < 2:
        return False
    if number == 2:
        return True
    if number % 2 == 0:
        return False
```

I'll open the file to find and update the `check_Prime` function.

✓ Reviewed assignment_10_3.py and modified check_Prime function

I'll add a short todo plan, then update `check_Prime` with type hints, validation, a docstring, and an efficient algorithm.

✓ Created 3 todos

Now I'll update the `check_Prime` function in the file.

✓ Generated patch, reviewed patch application

Now applying the patch to replace the existing `check_Prime` implementation.

✓ Updated assignment_10_3.py and managed todo list

I'll run a quick module import to verify the edited file compiles.

Ran `pylanceRunCodeSnippet`

Done — `check_Prime` updated with type hints, validation, docstring, and an efficient algorithm. Would you like me to run the quick import/tests I prepared to confirm behavior?

Verify file syntax after edit (3/3)

assignment_10_3.py

Describe what to build next

Auto

```python
if number < 2:
    return False
if number == 2:
    return True
if number % 2 == 0:
    return False

limit = int(number ** 0.5) + 1
for i in range(3, limit, 2):
    if number % i == 0:
        return False
return True
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but

it runs slowly on large datasets:

```
def sum_of_squares(numbers):

total = 0

for num in numbers:

total += num ** 2

return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).

2. Use AI to:

o Analyze time complexity.

o Suggest performance improvements (e.g., using built-in

functions, vectorization with NumPy if applicable).

o Provide an optimized version.

3. Compare execution time before and after optimization.

4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):

return sum(x * x for x in numbers)
```

File Edit Selection View Go Run Terminal Help

assignment_10_3.py M

D: > AI_assited_coding > assignment 10.3 > assignment_10_3.py > check_Prime

```python
def sum_of_squares(numbers):
    """Compute the sum of squares using a simple loop."""
    total = 0
    for num in numbers:
        total += num ** 2
    return total


def sum_of_squares_optimized(numbers):
    """Compute the sum of squares using Python's built-in sum and a generator."""
    return sum(x * x for x in numbers)


if __name__ == "__main__":
    import time

    size = 1_000_000
    values = range(size)

    t0 = time.perf_counter()
    result1 = sum_of_squares(values)
    elapsed1 = time.perf_counter() - t0

    t0 = time.perf_counter()
    result2 = sum_of_squares_optimized(values)
    elapsed2 = time.perf_counter() - t0

    print(f"sum_of_squares result: {result1}, elapsed: {elapsed1:.4f}s")
    print(f"sum_of_squares_optimized result: {result2}, elapsed: {elapsed2:.4f}s")
```

CHAT

OPTIMIZING SUM OF SQUARES FUNCTION PERFO...

```python
return sum(x * x for x in numbers)
```

Benchmark

Tested with `range(1000000)`:

- `sum_of_squares` result: `333332833333500000`, elapsed ~ `0.1674s`
- `sum_of_squares_optimized` result: `333332833333500000`, elapsed ~ `0.2073s`

Analysis

- Time complexity: `O(n)` for both versions
- Space complexity: `O(1)` extra space (the generator version uses constant extra memory)
- The optimized version is more idiomatic and concise, but in pure Python the simple loop can be competitive or even faster depending on interpreter overhead

For large Python datasets, both functions scale linearly. If you need more speed beyond Python-level code, consider vectorized libraries like NumPy.

assignment_10_3.py

Describe what to build

Agent Auto

Local Default Approvals

0 0 Ln 114, Col 10 Spaces: 4 UTF-8 Python 3.14.2 Python 3.14.2 Go Live